Simplifying Lean proofs in DiophantineConsistency

Source specific refactoring proposals

26 October 2026 Pinned static review

Start with reuse, not stronger automation. The clearest first edit is `half_base_bound_31` in `SelectiveCoefficientBounds`: it repeats the generic theorem immediately above it. Specializing that existing result at 31 produces a two-line candidate for the current 24-line proof body, with the same strict conclusion and assumptions. Next remove exact duplicated coordinate-packaging arguments and give long conjunction certificates named accessors. These changes make the intended mathematics more visible.

For the large generated proofs, the necessary DAG soundness, power, and degree APIs already exist. The useful next step is to route generated instances through them, retaining the literal circuit and public theorem contracts. The supplied conservative experiment replaces 193 individual power `ring` proofs with shared finite-label certificates while keeping all 1,192 public gate/power theorem header strings unchanged.

The measured circuit tradeoff is honest rather than spectacular: 660 fewer physical lines, but 2,360 more UTF-8 bytes in the final candidates. This is less repeated proof scripting, not byte compression, a smaller circuit, or a demonstrated speedup. More compact indexed facades are included as experiments, not advertised as drop-in replacements.

Status and trust. No Lean or Lake executable was available. **Every proposed Lean candidate is uncompiled.** This report supplies source-specific designs, exact pinned references, practical before/after edits, complete candidate artifacts, static evidence, and a replay roadmap. It does not claim an accepted patch, kernel verification, faster builds, or smaller elaborated proofs. The original source snapshot is unchanged.

Mathematical scope. Preserve the project's standard-axiom policy and every theorem's witness domain and hypotheses. This is not a proof of Con(ZFC), a consistency decision, or completion of the open polynomial/native-machine or internal-ZFC links identified in `README.md` lines 14–31.

Recommended first patch. Try the generic-bound specialization (Section 2.1), the finite-set support identities, and the sum-of-squares proof (Section 2.2). Replay them against the pinned baseline. Then compile all four compatibility tests for the shared nine-coordinate helper (Section 3.1). Leave representation changes and global automation until these small pilots establish the validation workflow.

Recommendation map

Priority 1 means a small interface-preserving pilot. Priority 2 is the next maintenance/refactoring batch. Priority 3 needs a measured consumer or abstraction experiment. Priority 4 is a later design option, not an immediate rewrite recommendation.

Rank	Change	Main caution	Section
1	Specialize the existing 31-variable coefficient bound	Compile the exact statement	2.1
1	Use sum-of-squares and finite-set identities	Conditional simp needs replay	2
1	Share nine-coordinate packaging and add named projections	Preserve slot 7 and positivity	3.1, 3.2
2	Reuse `run_power`, keep public wrappers	Certificate cost is unmeasured	4.2
2	Share source/window and modular-residue interfaces	Same source and parameters	3.3, 2.5
2	Generate pure audit wrappers and enforce axiom policy	Keep behavioral checks	5.2
3	Pilot indexed gates, shared degree profiles, and degree prefixes	Keep semantic cut points	4.3, 3.5, 4.5
4	Unify coefficient syntax or add typed circuit views	Representation bridges required	3.6, 4

The archive's before/after comparisons are measured text spans, not additive project-wide forecasts. In particular, the predecessor-exponent cleanup overlaps the 31-variable specialization, and the concise/explicit sum-of-squares proofs are alternatives. The 60-line tuple duplicate is an opportunity for shared maintenance, not a guaranteed net deletion after adding a helper and compatibility wrappers.

Contents

1 Scope and evidence **3**

2 Local changes worth trying first **4**

3 Simplify the interfaces between proofs **7**

4 Reduce generated proof repetition without changing the circuits **10**

5 Preserve trust and verify the migration **13**

A Reproducibility and artifact guide **16**

1 Scope and evidence

This review concerns shortening and simplifying the *existing* Lean proofs in `Computability/DiophantineConsistency`, not changing the polynomial construction, finishing its open mathematical links, or proving consistency. Each recommendation requires preservation of the public theorem's full type, including its parameters, witness domain, explicit hypotheses, and direction of implication.

1.1 The snapshot is fixed and inspectable

All source references target the immutable repository commit

`60e9c5d620c6c387d932876cfa2d9376f64ea56c`.

The archive includes the inspected source material and a manifest recording each selected blob's Git object hash, SHA256, repository path, and immutable source URL. The acquisition verified all 944 selected blob hashes. This fixes what was reviewed; it does not certify that Lean accepts a proposed replacement.

The active inventory is the `Lean/` tree at that commit. Its distinction between modules and checks is directory-based: some audit modules also reside in the module directory. Historical `con-zfc-diophantine-forms` copies and candidate-side provenance files are excluded from the active totals.

Source group	Files	Physical lines	Nonblank code lines
Namespace module directory	363	68,308	55,127
Umbrella import file	1	283	283
`checks/` directory	182	5,380	4,422
Active total	546	73,971	59,832

Table 1: Measured source size, not proof-term or compilation size. Code-line counts mask nested block comments, line comments, and string literals.

The namespace directory contains 4,626 lexical theorem or lemma declarations and 2,010 definitions or abbreviations. The inventory also flags 20 files with explicit generated-header patterns; this is a conservative heuristic, not a claim that every other file is handwritten. Declaration spans end at the next recognized declaration and are *not* exact proof-body measurements.

A second, naming-based classification separates 325 implementation-named files (66,574 lines) from 220 check or audit files (7,114 lines), plus the umbrella. It treats the 38 `*Checks` or `*Audit` files in the module directory as audits. These labels describe filenames, not Lean semantics.

1.2 What was and was not checked

The analysis used source inspection, comment-aware lexical checks, import-graph analysis, independently reproduced finite arithmetic checks where stated, source-diff measurements, and visual inspection of this PDF. Neither `lean` nor `lake` was available in the review environment. Consequently, **no proposed Lean snippet or patch in this archive has been compiled**, and no theorem-equivalence, kernel-replay, axiom-set, build-time, or proof-term improvement is reported as verified.

The delivered snapshot is not a standalone Lean project. Dependencies outside the inspected subtree and the complete Mathlib checkout are not included. Replay belongs in a *full* checkout of the pinned repository with its pinned dependencies; see Section 5.

2 Local changes worth trying first

These edits keep the mathematical interfaces in place. They are deliberately more specific than advice to replace proofs with `simp`, `aesop`, or `omega`. Each proposed simplification identifies an existing fact or a repeated invariant. All after blocks remain **uncompiled**.

2.1 Specialize the generic coefficient bound already in the file

Priority 1 and the best first pilot. `SelectiveCoefficientBounds.half_base_bound_31` at `SelectiveCoefficientBounds.lean` lines 203–232 reproves a result already available as `half_base_bound` at `SelectiveCoefficientBounds.lean` lines 166–199. The generic theorem has `m` bounded input digits and assumes

$$4(m+1)^2Hb^2 \le 2^t.$$

With $m = 31$, the scale is exactly $4096Hb^2$, and the specialized theorem's equality hypothesis supplies the required inequality.

The existing 24-line proof body repeats the sum bound, square bound, coefficient estimate, strict gap, and predecessor-power calculation. For example, it ends with this source excerpt:

```
have hb2 : 0 < (b : ℤ) ^ 2 := by positivity
have hgap : 2 * (H * (32 * (b : ℤ)) ^ 2) <
    4096 * H * (b : ℤ) ^ 2 := by
  nlinarith [mul_pos hH hb2]
have hpow : (2 : ℤ) ^ t = 2 * 2 ^ (t - 1) := by
  rw [← pow_succ']
  congr 1
  omega
rw [← hbase, hpow] at hgap
linarith
```

Replace the *whole proof body*, keeping the theorem statement intact:

```
refine half_base_bound w u p hH hp hb ht hu ?_ n
norm_num [hbase]
```

Measured candidate comparison: 24 physical proof-body lines become 2. This is a textual comparison, not an executed proof reduction. No new API is needed, and the strict conclusion `< 2 ^ (t - 1)`, the exact encoded polynomial, and all positivity/boundedness hypotheses remain. This is preferable to separately optimizing the duplicated proof's tactics.

2.2 State the sum of squares invariant directly

Priority 1. `ShortSystem.residualSum_zero_iff` at `ShortSystem.lean` lines 63–88 proves that five integer squares sum to zero exactly when all five residuals vanish. It currently makes five similar `nlinarith` calls, one for each square. The first is:

```
have h0 : (n0 - 1) ^ 2 = 0 := by
  nlinarith [sq_nonneg (n1 - 1), sq_nonneg (na - 1),
    sq_nonneg (ns - 1), sq_nonneg (nk * nt - 1),
    sq_nonneg (n0 - 1)]
```

Already present. The order API already provides `add_eq_zero_iff_of_nonneg`; the pinned Mathlib source generates this additive lemma from the multiplicative analogue with both factors at least one. Together with `sq_eq_zero_iff` and `sub_eq_zero`, it describes the proof's actual invariant.

The concise candidate replaces the complete 23-line body by two lines:

```
simp only [residualSum, add_eq_zero_iff_of_nonneg, add_nonneg,
  sq_nonneg, sq_eq_zero_iff, sub_eq_zero, and_assoc]
```

Its main replay uncertainty is whether conditional simplification discharges the nonnegativity premises as intended in this exact context. A more explicit fallback proves the generic five-square fact with the partial sums named:

```
have hab := add_nonneg (sq_nonneg a) (sq_nonneg b)
have habc := add_nonneg hab (sq_nonneg c)
have habcd := add_nonneg habc (sq_nonneg d)
rw [add_eq_zero_iff_of_nonneg habcd (sq_nonneg e),
  add_eq_zero_iff_of_nonneg habc (sq_nonneg d),
  add_eq_zero_iff_of_nonneg hab (sq_nonneg c),
  add_eq_zero_iff_of_nonneg (sq_nonneg a) (sq_nonneg b)]
simp only [sq_eq_zero_iff, and_assoc]
```

The archive also gives a direct, explicit source-statement fallback. Choose one proof, not both. `compactResidualSum_zero_iff` already reuses the original result, so it is not a second independent reduction. Do not import the heavier `Mathlib.NumberTheory.Dioph` module merely for an analogous sum-of-squares theorem. The lightweight order facts suffice.

2.3 Use finite set intersection instead of reproving membership

Priority 1 for a small isolated edit. `LowMaskCoding.sum_range_support` at `LowMaskCoding.lean` lines 39–46 first proves that filtering a range by membership in a contained set gives the set itself. The existing body is:

```
have hf : (range len).filter (fun j => j ∈ S) = S := by
  ext j
  simp only [mem_filter]
  exact and_iff_right_of_imp (fun hj => hS hj)
rw [← Finset.sum_filter, hf]
```

The proposed two-line body uses the existing finite-set API:

```
rw [← Finset.sum_filter, Finset.filter_mem_eq_inter,
  Finset.inter_eq_right.mpr hS]
```

`Finset.filter_mem_eq_inter` is explicitly present in the pinned Mathlib source. The same change applies to `MultiTargetCarry.selectedMask_eq` at `MultiTargetCarry.lean` lines 55–67, converting the pointwise bound `hp i hi` to range membership:

```
rw [← Finset.sum_filter, Finset.filter_mem_eq_inter,
  Finset.inter_eq_right.mpr
    (fun i hi => Finset.mem_range.mpr (hp i hi)),
  Finset.mul_sum]
```

The candidate files measure the changed spans as $5 \to 2$ and $6 \to 3$ physical lines; the latter is wrapped more generously here for readability. The mathematical gain is eliminating a second implementation of a standard set identity, not teaching the simplifier an extra global rule.

2.4 Remove the repeated predecessor exponent calculation

Priority 2. Seven local `hpow` proofs show `a ^ t = a * a ^ (t - 1)` under `1 ≤ t`. They share this three-line body:

```
rw [← pow_succ']
congr 1
omega
```

The existing power and natural-subtraction APIs give the proposed body:

```
rw [← pow_succ', Nat.sub_add_cancel ht]
```

The occurrences are in `MultiTargetCarry.lean` lines 40–43, `MultiTargetCarry.lean` lines 132–135, `MultiTargetCarry.lean` lines 226–229, `SelectiveCarry.lean` lines 89–92, `SelectiveCarry.lean` lines 111–114, `SelectiveCoefficientBounds.lean` lines 194–197, and `SelectiveCoefficientBounds.lean` lines 227–230. One disappears entirely if the 31-variable specialization is adopted, so its saving must not be counted twice. An extra project lemma is optional; it is not needed to write this one-line replacement. Keep the `ht` premise: natural subtraction is not cancellative without the bound.

2.5 Centralize the modular residue interface

Priority 2 for maintenance rather than raw compression. `CurrentClause2011.addition_residue` and `subtraction_residue` at `CurrentClause2011.lean` lines 27–39 are duplicated privately as `add_residue` and `sub_residue` at `CurrentReflectedGeometry2011.lean` lines 74–86.

The arithmetic only needs a divisor `L` of the ambient modulus `n`, not the concrete value 2011. A proposed low-level shared module can expose these exact interfaces:

```
-- Proposed generic statements; full proofs are in PortableCandidates.lean.
theorem addition_residue {n L : ℕ} [NeZero n] (hL : L | n)
    (b : ZMod n) (e : ℕ) :
    (b + (e : ZMod n)).val % L = (b.val + e) % L := by
  -- Proof omitted here; UNCOMPILED full candidate in the archive.

theorem subtraction_residue {n L : ℕ} [NeZero n] (hL : L | n)
    (k : ZMod n) (e : ℕ) :
    ((k - (e : ZMod n)).val + e) % L = k.val % L := by
  -- Proof omitted here; UNCOMPILED full candidate in the archive.
```

The full candidates use `ZMod.natCast_eq_natCast_iff` followed by `Nat.ModEq.of_dvd`, keeping the coercion normalization in one place. Keep the existing public names as concrete specializations. Do not make geometry import the clause layer merely to borrow these lemmas, and retain the positive-modulus instance. The helper-plus-wrapper cost means no large net line saving is asserted.

2.6 Smaller opportunities and a reason to keep an existing proof

The archive contains several further local experiments; they are useful after the higher-value reuse changes, not a mandate to minimize every proof.

- **Convolution factorization.** In `NativeCyclicConvolution.word_convolution`, the final distributivity step at `NativeCyclicConvolution.lean` lines 79–87 can be expressed using `Finset.sum_mul`, `Finset.mul_sum`, `pow_add`, and controlled associative/commutative simplification. The candidate changes that span from 9 to 3 lines. Preserve the preceding modular-congruence argument; a global `ring` call does not replace it.
- **Use the exact power fact.** At `NativeRadixDigits.lean` lines 54–57, replace a weaker power inequality, normalization, and `omega` with `Nat.one_lt_two_pow (by omega)`. The same existing lemma is already used at `NativeWindowCarry.lean` lines 63–63. The candidate changes the four-line `hr` declaration to one line.
- **Normalize a finite-switch boundary once.** At `ArithmeticFiniteSwitch.lean` lines 24–30, keep the equality case and then prove `tag < count + 1 ↔ tag < count` in its complement. This removes the redundant second case split; the supplied candidate span is 7 to 4 lines.
- **Do not prefer branch expansion just because it is shorter.** The optional `cellDigit_lt` candidate at `CurrentHighDigits2011.lean` lines 80–86 is two lines using `split_ifs` and `norm_num`, but

duplicates finite/numeral work already packaged by `highDigits_lt`. Keep the original unless actual readability and profiling favor the replacement.

Across these changes, retain explicit `simp only` boundaries for large kernel, stencil, and mask definitions. A project-wide collection of new `[simp]` tags would trade short source for unpredictable expansion in downstream proofs.

3 Simplify the interfaces between proofs

The strongest abstractions here follow repeated mathematical boundaries: coordinate packing, source-window extraction, and degree-layer estimates. They should reduce the number of arguments a maintainer has to understand, not merely hide an unchanged long proof behind a new name. The proposed helpers in this section are not existing declarations.

3.1 Prove the nine coordinate packaging equivalence once

Priority 1. The blocks in `SplitSourceExistence.lean` lines 98–157 and `GapSourceExistence66.lean` lines 53–112 are byte-identical: each is 60 lines and 2,292 UTF-8 bytes, excluding a trailing newline. They define positive and natural coordinate interpretations and prove the two existence equivalences. The scalar evaluator comes from the surrounding namespace; the packing argument does not inspect its arithmetic.

Already present. Gap already reuses Split's cofinal-cap lemmas at `GapSourceExistence66.lean` lines 12–15. That is a successful abstraction to preserve. The remaining duplicate is specifically the coordinate packaging.

Proposed. Parameterize one helper by the evaluator below, with a predicate `HasZero` expressing the same positive-coordinate existential. Retain the old public definitions and theorem types; specialize the generic proof inside their bodies.

```
-- Proposed helper boundary, UNCOMPILED.
abbrev Evaluator :=
  ℕ → ℕ → FreeCarrierRatio.FreeVars → ℕ → ℤ

def positiveVars (x : Fin 9 → ℕ) : FreeCarrierRatio.FreeVars :=
  ⟨x 2, x 3, x 4, x 5, x 6, x 7 - 1⟩

def naturalVars (x : Fin 9 → ℕ) : FreeCarrierRatio.FreeVars :=
  ⟨x 2 + 1, x 3 + 1, x 4 + 1, x 5 + 1, x 6 + 1, x 7⟩
```

This captures an important asymmetry. The positive tuple stores `b,g,h,s,w,C,i,j+1,n`, whereas the internal `FreeVars.j` is an unrestricted natural. In the natural tuple, slot 7 is `j`, not `j+1`. The generic theorem must not add `0 < v.j`, or omit positivity of `g` or `n`.

The supplied `architecture/NineCoordinatePackaging.lean` contains the complete proposed helper, including the successor/predecessor bridge. The companion compatibility file states four examples matching the two existing positive/natural existence interfaces in each source package. A conceptual replacement is:

```
-- Schematic specialization; helper name is proposed.
exact NineCoordinatePackaging.exists_natural_zero_iff_hasZero
  (GapSourceExistence66.value d a)
```

Tradeoff and gate. This is a good first abstraction because it moves no number theory and assumes no source correctness. Put it below both packages with only the needed foundational imports. Compile the

helper and all four compatibility examples first. The 60-line duplicate is an observed opportunity, not a net 60-line saving: the generic interface and compatibility wrappers also cost source text. The main gain is one maintained coordinate-bijection proof.

3.2 Give long conjunctions names without changing their definitions

Priority 1. `Carrier66.Canonical` is a seven-part conjunction at `Carrier66.lean` lines 153–163. A consumer in `canonical_strong_error` reaches a fact through six positional projections at `Carrier66Source.lean` lines 167–191. The fragility is not proof search; it is that a new conjunct can silently make every positional access harder to read or maintain.

```
-- Current source, Carrier66Source.lean:180.
rw [hc.2.2.2.2.2.1, sub_self] at hid

-- Proposed accessor, after proving Canonical.Z_eq_delta.
rw [hc.Z_eq_delta, sub_self] at hid
```

Add derived projection lemmas such as `Canonical.Z_eq_delta`, `Canonical.index_eq_one`, and `Canonical.rank_pos`. Initially keep `Canonical` itself unchanged, so existing destructuring patterns and constructors continue to work. A later `Prop` structure is optional, with a proved equivalence; it is not required to obtain readable accesses.

The same restrained approach can expose `PortBounds.beta_pos` and a small core-positivity interface. Reuse `Carrier65.scales_pos` at `Carrier65.lean` lines 175–183 rather than turning derived positivity into new assumptions. The existing `CurrentPolynomialWindow2011.Facts` record at `CurrentPolynomialWindow2011.lean` lines 27–68 is the local precedent for coherent named fields.

3.3 Share one coherent source and scalar window certificate

Priority 2. The low-digit and high-digit consumers repeat the same restoration and normalization pipeline. Compare `polynomial_zero_reconstructs` at `CurrentSparseDigits2011.lean` lines 111–138 with `polynomial_zero_mask_test` at `CurrentHighDigits2011.lean` lines 135–159. Their 13-line preambles turn a polynomial zero into a `Carrier65.value` zero, restore a canonical parent, call the scalar-window theorem, rewrite the restored core, and normalize powers into the current radix. The fuller `polynomial_zero_window` repeats the parent/source bridge because it also needs the actual source assignment (`CurrentPolynomialWindow2011.lean` lines 78–119).

Already present. `Carrier66Source.canonical_window` at `Carrier66Source.lean` lines 376–394 already proves the generic scalar window. `positive_carrier_zero_source` at `Carrier66Source.lean` lines 398–416 already supplies a source lift. The proposal is to compose and expose these results once, not to replace them with a second theory of windows.

Proposed. First expose a direct positive `Carrier65` zero-to-source certificate containing the natural source, positivity, rows, preserved outer coordinates, and the existing `sourceQ_eq`, `gap_eq`, and `nativeRank_eq` equalities. Then add one current-mask specialization that normalizes width/radix and presents both mask tests together.

```
-- Interface sketch only; names and record are proposed.
obtain ⟨N, hN, facts⟩ := current_zero_window_facts K hK v hv hz
-- Low decoder consumes facts.C_pos, facts.C_split, facts.low_mask.
-- High decoder consumes facts.high_mask.
-- Both use the same facts.q_window and facts.J_window.
```

The record must carry one *shared* `N` and source assignment. Separate existential choices for different fields would weaken the usable interface. Keep source lift, scalar window, and later carry/digit facts in distinct layers so the low-level helper does not import its consumers.

Tradeoff and gate. Preserve the high port's single beta shift and both origin corrections, low `+1` and high `-1`. The public theorem must still derive the certificate from positivity and the actual polynomial zero. Adding a canonical-parent, source-zero, no-carry, or decoded-window hypothesis would shorten the proof by changing its theorem, not simplify it.

3.4 Put the current base interface below completeness

Priority 3, after the window interface. `CurrentCarrierArithmetic2011.lean` lines 1–2 imports both `CurrentMasks2011` and `ThirteenCarrierComplete`. Its low/high digit clients need the four-port bounds, `polynomial65`, and the carrier-to-source arithmetic bridge, not the thirteen-coordinate completeness layer. The latter's definitions and equivalences occur at `CurrentCarrierArithmetic2011.lean` lines 45–132.

Extract a proposed `CurrentCarrierBase2011` below completeness, importing `CurrentMasks2011` and `Carrier66Source`. Keep the old arithmetic module as a compatibility facade and redirect only the simpler clients to the base. The static local import closure changes from 37 existing modules to 31 existing modules plus the proposed base. The five avoided existing dependencies, apart from replacing the old facade itself, are `SignedPellResidue`, `ThirteenCarrier`, `ThirteenCarrierComplete`, `ThirteenCarrierSoundness`, and `ThirteenFreshSeed`.

This is a measured graph comparison, *not* a build-time improvement. External transitive imports were not traversed. Preserve the facade's names and avoid a dependency cycle by never importing the digit/window consumers back into the base.

3.5 Abstract the degree profile but retain sharp hypotheses

Priority 3. The source-positivity arguments in `SplitWordBounds.lean` lines 76–123 and `GapWordBounds66.lean` lines 31–78 share the same numerical proof structure. Their upper-scale calculations likewise appear at `SplitWordBounds.lean` lines 149–200 and `GapWordBounds66.lean` lines 81–132. Gap already aliases several source and packing definitions and imports common geometric lemmas at `GapWordBounds66.lean` lines 11–20; preserve that reuse.

The proposed generalization is a small degree profile (d_0, d_1, d_2) with the two actual gap inequalities

$$d_1 < d_2 + 116, \qquad d_0 < d_2 + 232.$$

Retain the leading-coefficient condition `D2.coeff d2 = 1`, each degree and height bound, the scale $B = 4096Hb^2$, and $B^{116} \le U$. Prove lower-layer domination and positivity once. Instantiate the profile with $(1719, 2027, 1929)$ for Split and $(1627, 1941, 1850)$ for Gap.

```
-- Schematic proposed API, not an existing theorem.
exact degreeProfile_negative_layers hk.profile hH hb hscale hU
  (by decide : 1941 < 1850 + 116)
  (by decide : 1627 < 1850 + 232)
```

Only after this lower-bound interface proves useful should it absorb the upper packing estimate. A target exponent $d_2 + 233$ gives the existing concrete exponents 2162 and 2083. Do not replace either sharp bound by a larger one simply to make the abstraction easier.

Tradeoff and gate. Gap's `KernelBounds` is not a stronger version of Split's: its leading coefficient is at 1850 rather than 1929. Compare `SplitWordBounds.lean` lines 20–27 and `GapWordBounds66.lean` lines 22–29. An attempted coercion between them would be mathematically wrong. Store degree data and elementary inequalities, not the final conclusions. Symbolic exponents can also increase elaboration cost, so benchmark both concrete specializations, not only the new theorem.

3.6 Remove duplicated coefficient expressions in stages

Priority 4. The region `NativeNinePolynomial52.lean` lines 93–404 contains 22 `mapRowCoefficient*` helpers and their dispatch, with repeated `hleft`/`hright` spellings of original and mapped coefficients. The authoritative table is only `NativeQuadraticParent.lean` lines 155–177. Gap has analogous transport cases at `GapNinePolynomial66.lean` lines 99–283.

The reviewed native table uses fixed port indices, integer numerals, negation, ring addition, subtraction and multiplication, and the fixed natural exponent 2. Its branches compare a fixed pair index, not ring values. That specific fragment admits a ring-homomorphism interpretation. It does not justify transporting arbitrary natural evaluators, truncated subtraction, division, or `toNat`. The natural exponent subtraction in `kernelValue` at `NativeQuadraticParent.lean` lines 64–64 is fixed layout metadata and must remain fixed.

Already present. Generic evaluator naturality already exists in `ArithmeticDAG.lean` lines 48–103; the carrier and polynomial files already use `value_map` and `eval_polynomial`, for example `Carrier65.lean` lines 155–167 and `NativeNinePolynomial52.lean` lines 406–454.

Proposed. Try the cheaper change first: a focused local simp interface or generated case tactic that keeps the coefficient table and removes the duplicated intermediate expressions. Only if measured maintenance savings warrant it, encode these small coefficients once as computable ring syntax and prove its evaluator naturality once. Reuse the existing DAG evaluator where it helps; do not force a tiny coefficient into a large framework.

Tradeoff and gate. Keep a proved equality to the old authoritative table, preserve the general `CommRing` theorem, and retain row order, support, repeated-term accumulation, and corrections. Do not define both the source and certificate from the same replacement table and treat reflexivity as independent verification. Replacing the ordinary integer evaluator with noncomputable polynomial evaluation would cross the intentional boundary documented at `NativeNinePolynomial52.lean` lines 9–13.

4 Reduce generated proof repetition without changing the circuits

The largest source concentration is the pair `SplitCircuitSteps59` and `GapCircuitSteps66`: 6,567 physical lines, comprising 999 gate wrappers and 193 power lemmas. This is a strong source-maintenance target, but it does not mean 1,192 independent mathematical ideas need to be replaced. The gate family already consists of instances of one generic theorem.

4.1 Use the semantic infrastructure already present

The project has the key reusable results now. A proposal to introduce its first generic DAG soundness or degree checker would miss these interfaces:

Gate and register equations
`run_node_equation`, followed by `run_eq_of_node_equations` and `value_eq_of_node_equations`, at `ArithmeticDAGSemantics.lean` lines 58–116.
Power labels and circuit rewrites
`run_power` at `ArithmeticDAGPowers.lean` lines 24–42, and `value_eq_of_powerRewrite` at `ArithmeticDAGPowers.lean` lines 86–131.
Degree certificates
`DegreeCertificate` at `ArithmeticDAGDegree.lean` lines 21–23, with `value_degree_le` and `polynomial_degree_le` at `ArithmeticDAGDegree.lean` lines 65–90.

Existing generated module	Total lines	Gate wrappers	Power lemmas
`SplitCircuitSteps59`	3,445	515	107
`GapCircuitSteps66`	3,122	484	86

Table 2: Gate spans are Split lines 16–2590 and Gap lines 16–2435. Power spans are Split lines 2591–3444 and Gap lines 2436–3121.

The measured gate spans occupy 4,995 lines and the power spans 1,540, including blank lines. Together they cover 6,535 of the 6,567 lines. These figures locate the repetition; they are not a deletion budget, because clients currently use the generated names.

4.2 Consolidate power proofs while retaining every public theorem

Priority 2 and the safest circuit experiment. Preserve the gate wrappers, every public theorem header, and the literal DAG. Replace the individual pure-power `ring` proofs with a finite exponent-label certificate and the existing `run_power` theorem.

The current `GapCircuitSteps66.natural_power_25` at `GapCircuitSteps66.lean` lines 2442–2448 is:

```
theorem natural_power_25 (p : Fin 11 → R) (x : Fin 9 → R) :
    naturalRegister p x 25 = naturalRegister p x 20 ^ 4 := by
  rw [natural_step_25]
  simp only [natural_power_24]
  ring
```

After adding the shared private certificate and generic specialization, the candidate keeps that public proposition and changes only its proof:

```
theorem natural_power_25 (p : Fin 11 → R) (x : Fin 9 → R) :
    naturalRegister p x 25 = naturalRegister p x 20 ^ 4 :=
  natural_power_all p x ⟨25, by decide⟩ (by decide +kernel)
```

The label table uses zero to mean *not certified as a power*, not exponent zero. The base has label one: register 36 in Split, register 20 in Gap. Every positive non-base label belongs to a multiplication node whose positive operand labels add to the result label. The existing topology argument supports induction over register number, uniformly for every `CommRing`.

For example, the proposed closed Gap obligation is:

```
private theorem naturalPowerSteps : ∀ i : Fin 484,
    i.val ≠ 20 → naturalPowerLabel i.val ≠ 0 →
      PowerStep GapCircuit66.naturalNodes naturalPowerLabel i.val := by
  decide +kernel
```

The full `natural_power_all` proof invokes `run_power` and this certificate. The archive contains the full label tables and complete API-preserving candidate modules, not just the abbreviated listing here.

Final candidate	Physical lines	UTF-8 bytes	Per-power `ring` calls
Split before	3,445	169,808	107
Split candidate	3,074	171,042	0
Gap before	3,122	153,802	86
Gap candidate	2,833	154,928	0

Table 3: Static measurements of the final, audit-inclusive, uncompiled candidate files. All 1,192 public theorem header strings were compared byte-for-byte; this is not an elaborated theorem-type comparison.

The result is 660 fewer physical lines but 2,360 more bytes. It demonstrates less tactic repetition, not byte compression or a speedup. Much of the line change removes proof-body layout. The cost of one finite certificate could exceed that of the old local proofs; benchmark before adoption, and split it into checked chunks if a monolithic check is costly.

There is an additional reuse opportunity, not a new data format: the Split labels extracted for this review are exactly the integer list already stored as `SplitJointPower59.natural_oldLabelsData` at `SplitJointPower59.lean` lines 64–78. Extract that table and its certificate into a small shared parent module rather than duplicate it or import the joint child circuit merely to obtain parent metadata.

4.3 Offer an indexed facade before removing named wrappers

Priority 3. Each gate wrapper specializes `run_node_equation`. Instead of materializing every instance, expose one theorem indexed by `Fin nodeCount`. The proposed Gap facade is:

```
theorem natural_step (p : Fin 11 → R) (x : Fin 9 → R) (i : Fin 484) :
    naturalRegister p x i.val =
      (GapCircuit66.naturalNodes.getD i.val (.literal 0)).evalAt p x
        (naturalRegister p x) :=
  run_node_equation p x GapCircuit66.naturalNodes
    GapCircuit66.natural_topological i.val
    (by simpa only [GapCircuit66.natural_length] using i.isLt)
```

The supplied compact facades are 94 lines for Split and 92 for Gap. They contain the register evaluator, indexed step theorem, complete label data, power theorem, concrete examples, and axiom probes. **They are not drop-in replacements**: their namespaces differ and the 1,192 old named theorems are absent. Their size is not a measured project-wide saving.

Start with one genuine consumer such as `GapCircuitRegisters66.natural_baseRegister` at `GapCircuitRegisters66.lean` lines 18–22. Check that a concrete indexed instance rewrites without unfolding the whole evaluator. Then retain useful semantic boundary names such as `natural_kernel0Register`, `natural_sourceHighRegister`, and `natural_sourceRankRegister` while gradually retiring only the mechanical wrappers.

The direct Split consumers are Registers59 and Translation59. The direct Gap consumers are Degree66, Tail66, Translation66, and Registers66; recipes depend on them transitively. Rebuild the reverse dependency closure, not just these direct imports. In particular, the long, selective rewrite lists at `GapCircuitRegisters66.lean` lines 30–40 maintain deliberate cut points in a shared expression graph. Replacing them with `simp [all definitions]` can expand shared subexpressions repeatedly.

4.4 Certify semantic blocks instead of globally normalizing the DAG

Priority 3, only after a consumer pilot. The existing Gap Registers file already establishes 13 semantic register facts at `GapCircuitRegisters66.lean` lines 18–94, and the recipe composes them at `GapCircuitRecipe66.lean` lines 15–211. This is the right proof structure. Generalize it only where the kernel-block argument truly repeats.

A possible next layer is a block certificate with an input/base register, sparse coefficient/exponent data, and an output register. A local symbolic-value table can use `value_eq_of_node_equations` to establish the block's meaning. It must also prove that the certified block embeds into the *existing full DAG* and has the claimed output register. Proving a freshly generated, similar-looking circuit correct is not enough.

Keep the independently transcribed high-level recipe as the specification. Defining the specification to be the same generated DAG would make agreement vacuous. `SplitJointPower59` already demonstrates

genuine certificate reasoning between distinct literal circuits: the natural parent has 515 nodes and the joint child 498. Those are intentionally different objects.

4.5 Share the ShortSystem degree prefix before changing representation

Priority 3. The two generated degree sections in `ShortSystem` are at `ShortSystem.lean` lines 565–842 and `ShortSystem.lean` lines 844–1115. They contain respectively 96 and 94 local `let`/`have` pairs and share an exact 224-line prefix beginning at the first local binding.

The least disruptive change is a common core-degree lemma exposing only the boundary facts needed by the two output tails. A larger alternative uses the existing `ArithmeticDAGDegree` checker for a shared literal prefix and two residual/output tails. Either way, retain the bridges to `literalValue`/`compactLiteralValue` and the original polynomial values at `ShortSystem.lean` lines 348–359 and `ShortSystem.lean` lines 487–498.

The public bound is degree *at most* 68, not exact degree 68. Preserve that bound. A syntactic degree checker can miss cancellations; the explicit warning at `GapCircuitDegree66.lean` lines 67–67 documents this limitation elsewhere in the same development. A weaker bound is not an acceptable substitute merely because it is easier to certify.

4.6 Treat typed circuit views as a later design option

Priority 4. `ArithmeticDAG.Node` already uses `Fin` for input and fixed-leaf indices, but natural numbers for operand registers (`ArithmeticDAG.lean` lines 11–18). Invalid register access returns zero (`ArithmeticDAG.lean` lines 79–81); a separate topology certificate excludes invalid edges for well-formed circuits (`ArithmeticDAG.lean` lines 105–112).

A small checked-circuit wrapper can bundle the existing list, its topology proof, and a bounded output index. A dependent `SafeNode` view with operands in `Fin earlier` is possible later. In both cases prove that erasing the view returns *exactly* the original literal list in its original order. That equality is the bridge preserving operation counts, liveness, data audits, and the existing semantics.

This does not verify a Python importer or prove specification equivalence by itself. Dependent equalities can increase elaboration cost. It is lower priority than the direct reuse of `run_power`, and no timing benefit is claimed. In particular, none of the circuit proposals changes the original natural operation counts, 480 for Split at `SplitCircuit59.lean` lines 1246–1246 and 450 for Gap at `GapCircuit66.lean` lines 1186–1186.

5 Preserve trust and verify the migration

A shorter proof is acceptable only if it proves the same proposition about the same objects under the same trust policy. Static checks narrow the search for good edits; they do not close that obligation. This section separates the evidence obtained here from the gates still required before adoption.

5.1 What the static audit actually establishes

The comment/string-masked scan covers all 546 active Lean files. It found no code-token hits for `sorry`, `admit`, `sorryAx`, explicit `axiom` declarations, `native_decide`, `decide +native`, `bv_decide`, native-reflection symbols, or `unsafe`/`extern`/`implemented_by`. It counted 2,163 `#print axioms` commands, zero `#guard_msgs` commands, and 525 `decide +kernel` occurrences.

These are *lexical* findings. The scanner is not a Lean parser, and the result says nothing conclusive about macro expansion, imported dependencies, or the elaborated proof's transitive axiom set. External imports

include `Diophantine`, `BusyBeaver.ZFC`, `BoundedZFCConsistency`, `FirstOrder`, and Mathlib. Nor does the word `Native` in a module name indicate use of Lean's native evaluator.

The finite circuit analysis independently checked list lengths, input and fixed-leaf bounds, strictly earlier references, and every positive exponent transition. An altered exponent and a forward/non-earlier edge were rejected in disposable Python fixtures for both circuits. These are useful data-consistency and checker-coverage tests, **not Lean proofs**.

The validation helpers passed 16 Python tests, including malformed/missing axiom-report cases. The shell replay harness passed syntax checking and its unavailable-toolchain branch returned the intended exit code 69. Its actual Lean success branch was not run. Neither this evidence nor a successful PDF build is a candidate elaboration result.

5.2 Turn the existing audit output into a failing policy gate

The repository documents the intended standard-axiom policy and reproduction commands at `README.md` lines 908–939. Preserve it: an accepted endpoint may depend on a subset of

$$\{\texttt{propext}, \texttt{Classical.choice}, \texttt{Quot.sound}\}.$$

Reject every other axiom rather than checking only a few known bad names. This catches custom assumptions, `sorryAx`, and native-evaluation dependencies irrespective of their spelling in a particular Lean release.

`#print axioms` reports dependencies but does not itself reject a violation. No failure-producing axiom policy was found in the inspected committed locations: the pinned repository root has no `.github` entry, and the inspected root scripts contain no such gate. This does not establish that external or private CI is absent.

The supplied `validation/check_axiom_log.py` rejects missing expected targets, an empty target list, malformed/incomplete reports, unexpected axioms, compiler-error text, and sorry warnings. It requires an explicit endpoint list and accepts wrapped axiom lists and apostrophes in names. Its successful status is deliberately `PASS_LOG_ONLY`: it does not authenticate the log or establish a successful compiler exit. Capture merged stdout/stderr and preserve Lean's exit code separately.

Apply the policy to both baseline and candidate endpoints, and to new helpers on which they depend. Retain proof-producing finite methods, including the project's `decide +kernel` convention. Replacing them with `native_decide` to shorten source would change the trust boundary, not achieve a like-for-like simplification.

5.3 Consolidate pure audit wrappers but keep the regression proofs

Priority 2 for audit maintenance. Of the 220 filename/path-defined check or audit files, only 60 consist purely of import/open/namespace/end commands and `#print axioms`. Those wrappers occupy 739 lines and 33,098 bytes and contain 530 print sites. A declarative manifest of fully qualified endpoint names, required imports, test group, and axiom policy could generate their scaffolding.

This does not eliminate 739 lines of mathematical proof: the names and all 530 checks must survive. The 217 already fully qualified printed targets observed in the current source are all distinct, so no measured duplicated target removal is claimed. Resolve unqualified names in their existing namespace before centralizing them. Keep small audit shards for targeted rebuilds rather than defaulting to one expensive umbrella test.

Do not delete the other checks as boilerplate. The full check/audit group contains **835 examples and 85 theorem or lemma declarations**. In particular:

- `NativeCyclicCarryAudit.lean` lines 34–70 tests wraparound, a reset, and genuine nonuniqueness; the last protects against an invalid uniqueness assumption about cyclic carries.
- `CurrentHighDigits2011Checks.lean` lines 10–36 tests boundary cells and origin subtraction. Its check at `CurrentHighDigits2011Checks.lean` lines 48–54 preserves the implication from polynomial zero rather than assuming the desired mask property.
- `Carrier65Checks.lean` lines 18–49 fixes coordinate slots and negative-tau behavior.

The selected `EndpointAudit.lean` in this archive contains 35 additional target probes. It is uncompiled and not exhaustive; it does not replace these existing behavioral checks.

5.4 An incremental acceptance roadmap

Stage 0 Establish the baseline. Use a full checkout at the pinned commit, Lean 4.32.0, and Mathlib `81a5d257c8e410db227a6665ed08f64fea08e997`. Build the original target and run its existing audits, recording exact source/configuration hashes, compiler exits, and raw logs. The presence of an old `.olean` file is not evidence of a fresh trustworthy build. The focused archive lacks the full dependency tree and cannot substitute for this baseline.

Stage 1 Pilot the local reuse edits. Start with `half_base_bound_31`, finite-set support, and the sum-of-squares proof. Keep every public declaration statement unchanged. Then try the tuple helper and named accessors. Compile new generic modules first, compatibility modules second, and every affected reverse dependency afterward. This stage establishes the workflow before touching generators.

Stage 2 Test the conservative circuit certificate. Compile the API-preserving power candidates against the unchanged literal DAGs. Confirm all original public endpoints and recipe consumers, not just the new private theorem. Compare the baseline and candidate axiom sets and record actual certificate cost. Only then pilot the indexed facade on one consumer. Update the authoritative generator if adopted; hand-editing only generated output would lose the refactor on regeneration.

Stage 3 Extract the shared semantic interfaces. Introduce the source/scalar-window certificate, refactor the two simple digit clients, then the full window proof. Perform the base-module import split only after that interface is stable. Next parameterize lower-layer degree profiles, retaining concrete wrappers and all sharp constants. Try the smaller coefficient-map cleanup before an AST migration.

Stage 4 Accept or reject on evidence. Build the explicit main library target, the full affected import closure, and all 220 existing check/audit files. Compare exported types, preserved definitions or semantic bridges, axiom sets, boundary/negative tests, and generator reproducibility. Keep the old names as aliases until consumers pass. Reject a shorter edit if it hides a changed assumption, worsens maintainability, or creates unacceptable replay cost.

5.5 The invariants that a line-count target must not erase

1. **Statement and definition preservation.** Keep fully qualified public names, universe parameters, implicit/explicit binders, typeclasses, hypotheses, and conclusions. Identical theorem text is insufficient if referenced definitions change; freeze them or prove an explicit bridge to the old semantics.
2. **Concrete representation preservation.** For circuits retain input dimensions, fixed-leaf dimensions, register bases, outputs, literal node order, topology, liveness, zero-default semantics, and operation-count definitions. For witness maps retain coordinate order, positivity, source binding, row permutations, and origin corrections.
3. **Strict arithmetic interfaces.** Preserve natural-subtraction side conditions, strict digit/carry bounds, and the exceptional all-maximum cyclic representation. Do not change `word < radix^n - 1` into a non-strict inequality, or assume local no-carry when the proof establishes a quotient fixed point instead.

4. **Independent specifications.** The certified data must remain linked to the actual source polynomial, literal DAG, and high-level recipe. A data hash proves identity, not correctness; a table defined from itself proves neither independent transcription nor specification agreement.

Several useful simplifications are already present. The generic cyclic-carry API and the periodicity/mod-four argument in `NativeFieldCycles2011.lean` lines 49–95 should be reused, not presented as new refactors. Likewise the existing source-window, DAG soundness, power, and degree interfaces are assets of the baseline, not deliverables of this review.

5.6 Reproduction commands and their limits

The source-only checks below can be rerun from the extracted archive root:

```
python3 verify_evidence.py
python3 validation/test_tools.py
python3 provenance/verify_sources.py
```

The upstream baseline commands, to run later in a configured full checkout, include:

```
lake build DiophantineConsistency
lake env lean Computability/DiophantineConsistency/Lean/checks/ZFCAxioms.lean
lake env lean Computability/DiophantineConsistency/Lean/checks/CompilerAxioms.lean
```

The supplied future replay harness takes a full clean pinned checkout, an existing direct toolchain binary directory, and optionally a new output directory. It installs nothing and keeps candidate outputs outside the checkout:

```
bash validation/replay_candidates.sh \
  /path/to/full/ProveIt /path/to/lean-4.32.0/bin /path/to/new-output
```

It targets Linux/WSL with GNU utilities. Even if it later succeeds, it only establishes candidate elaboration and selected axiom checks, not a complete downstream integration build or full regression pass. Its prerequisites and the complete acceptance contract are in `validation/README.md`.

When measuring performance, hold the toolchain, dependencies, machine, parallelism, and cache state fixed. Record clean and incremental wall time, peak resident memory, and generated bytes separately. An `.olean` file's size is not proof-term size. No runtime, heartbeat, memory, kernel-proof-size, or build-time saving was measured in this review.

A Reproducibility and artifact guide

The archive deliberately separates original material, review evidence, uncompiled candidates, and this report. Start with the PDF or its TeX source, then use the following supporting material when selecting an implementation pilot. None of the candidate files was applied to the source snapshot.

`report/`
This report's TeX sources, PDF, build script, and typesetting requirements. The report is editable; `bash build.sh` rebuilds it with XeLaTeX. Its successful PDF build is independent of the unavailable Lean replay.
`sources/`
The inspected, unmodified repository material. This is a focused review snapshot, not a replacement for a full repository checkout.
`provenance/`
The selected-source manifest, source-fetch records, SHA checks, and scripts for materializing and authenticating the snapshot.

`inventory/`
Per-module source metrics, declaration-span triage, import edges, scope-aware totals, and the inventory narrative. CSV data are suitable for sorting; JSON records preserve the measurement definitions.
`tactics/`
Local proof candidates, source replacement excerpts, pinned upstream API evidence, and source-diff measurements. The portable experiment imports `Mathlib`; that broad import is not a production recommendation.
`architecture/`
Abstraction and dependency proposals, coordinate-packaging candidates, compatibility tests, and the static import-graph/repetition evidence.
`circuits/`
The two indexed circuit facade candidates, extracted exponent labels, generated-proof census, static arithmetic checks, and analysis script.
`validation/`
The comment-aware static audit, endpoint list and uncompiled audit module, axiom-log checker, validation tests, and replay instructions. These helpers are not substitutes for the actual Lean compiler and kernel.

A.1 Three measurements that must stay distinct

Physical source size counts newline-delimited text. A generated list can be wrapped in many ways without changing its information content, so a line reduction should be accompanied by byte counts and an explanation of which old names or data it retains. **Maintenance surface** concerns how many independently maintained arguments, public declarations, and duplicated normal forms exist. It can improve even when compatibility wrappers prevent a net line reduction. **Kernel and elaborator cost** must be measured by building both revisions in the same pinned environment; this review did not measure it.

The static scripts are explicit about scope. Lexical token counts suppress comments and strings but do not parse Lean. Import reachability follows only the selected local source graph; it does not model Lake's glob selection or external modules. Certificate arithmetic checks validate finite numerical relations independently but do not establish that the corresponding Lean declaration elaborates or uses only the intended axioms.

A.2 Reference conventions

All source links in the report are immutable GitHub blob links at the full commit above. Unless a path is explicitly rooted elsewhere, file references are relative to `Computability/DiophantineConsistency/Lean/DiophantineConsistency/`. Line ranges refer to the original snapshot, not to candidate files. Supporting Markdown reviews contain additional anchors and full reasoning; the main report selects the changes with the clearest maintenance payoff.

Bottom line. Preserve the literal objects and theorem contracts, reuse the project's existing semantic infrastructure, and remove repeated presentation proofs in small replayable steps. A shorter source file is a useful result only when the same theorem still checks under the same trust boundary and remains easier to maintain.